

Chandy-Lamport Algorithm for Distributed Debugging

IMPLEMENTATION:

```
class Process:
    def __init__(self, pid):
        self.pid = pid
        self.local_state = 0
        self.outgoing = []
        self.recorded = False
        self.channel_state = {}
        self.recording_channels = {}

    def add_channel(self, process):
        self.outgoing.append(process)

    def send_message(self, receiver, message):
        print(f"P{self.pid} sends '{message}' to P{receiver.pid}")
        receiver.receive_message(self.pid, message)

    def receive_message(self, sender_pid, message):
        print(f"P{self.pid} received '{message}' from P{sender_pid}")

        # If snapshot recording and still recording this channel
        if self.recorded and self.recording_channels.get(sender_pid, False):
            self.channel_state[sender_pid].append(message)

    def initiate_snapshot(self):
        print(f"\n--- P{self.pid} initiates Snapshot ---")
        self.record_state()

        # Send marker to all outgoing channels
        for process in self.outgoing:
            process.receive_marker(self.pid)
```

```

def receive_marker(self, sender_pid):
    if not self.recorded:
        print(f"P{self.pid} received FIRST marker from P{sender_pid}")
        self.record_state()

        # Mark sender channel as empty
        self.recording_channels[sender_pid] = False

        # Send marker to all outgoing channels
        for process in self.outgoing:
            process.receive_marker(self.pid)
    else:
        print(f"P{self.pid} received SECOND marker from P{sender_pid}")
        self.recording_channels[sender_pid] = False

def record_state(self):
    self.recorded = True
    print(f"P{self.pid} records local state = {self.local_state}")

    # Start recording all incoming channels
    for process in processes:
        if process.pid != self.pid:
            self.recording_channels[process.pid] = True
            self.channel_state[process.pid] = []

def print_snapshot(self):
    print(f"\nSnapshot at P{self.pid}")
    print(f"Local State: {self.local_state}")
    print("Channel States:")
    for sender, messages in self.channel_state.items():
        print(f"    From P{sender}: {messages}")

# ----- MAIN PROGRAM -----

# Create processes
P1 = Process(1)
P2 = Process(2)
P3 = Process(3)

processes = [P1, P2, P3]

# Define channels (Ring structure)
P1.add_channel(P2)
P2.add_channel(P3)
P3.add_channel(P1)

# Normal message passing before snapshot
P1.send_message(P2, "M1")
P2.send_message(P3, "M2")

# Initiate snapshot at P1
P1.initiate_snapshot()

# Messages during snapshot
P2.send_message(P3, "M3")
P3.send_message(P1, "M4")

# Print snapshot result
P1.print_snapshot()
P2.print_snapshot()
P3.print_snapshot()

```

OUTPUT:

```
P1 sends 'M1' to P2
P2 received 'M1' from P1
P2 sends 'M2' to P3
P3 received 'M2' from P2

--- P1 initiates Snapshot ---
P1 records local state = 0
P2 received FIRST marker from P1
P2 records local state = 0
P3 received FIRST marker from P2
P3 records local state = 0
P1 received SECOND marker from P3
P2 sends 'M3' to P3
P3 received 'M3' from P2
P3 sends 'M4' to P1
P1 received 'M4' from P3
```

```
Snapshot at P1
Local State: 0
Channel States:
| From P2: []
| From P3: []
```

```
Snapshot at P2
Local State: 0
Channel States:
| From P1: []
| From P3: []
```

```
Snapshot at P3
Local State: 0
Channel States:
| From P1: []
| From P2: []
```